K.R. MANGALAM UNIVERSITY

Database Management Assignment File (ENCS205)

Bachelor of Technology

in

Computer Science and Engineering

Submitted by:

Divesh (2401010077)

B. Tech CSE (Core) – 4th Semester

Section: A

Submitted To:

MS. Mansi Kajal

School of Engineering & Technology
K. R. MANGALAM UNIVERSITY
Sohna, Haryana 122103, India

Jan-2026

1. **Consider following databases and draw ER diagram and convert entitiesand relationships to relation table for a given scenario:**

Aim: To analyze the requirements for a College Database, design an Entity-Relationship (ER) Diagram representing Students, Sections, Subjects, and Marks, and convert the conceptual model into a Relational Database Schema.

COLLEGE DATABASE:

STUDENT (USN, SName, Address, Phone, Gender)

SEMSEC (SSID, Sem, Sec)

CLASS (USN, SSID)

SUBJECT (Subcode, Title, Sem, Credits)

IAMARKS (USN, Subcode, SSID, Test1, Test2, Test3, FinalIA)

PROCEDURE:

1. The entities **STUDENT, SEMSEC, SUBJECT, and IAMARKS** are identified and represented using rectangles.
2. The attributes are represented using ellipses:

STUDENT: USN (Primary Key), SName, Address, Phone, Gender

- USN is underlined as it is the Primary Key.
- Phone is shown as a single-valued attribute.

SEMSEC: SSID (Primary Key), Sem, Sec

- SSID is underlined as the Primary Key.

SUBJECT: Subcode (Primary Key), Title, Sem, Credits

- Subcode is underlined as the Primary Key.

IAMARKS: USN, Subcode, SSID, Test1, Test2, Test3, FinalIA

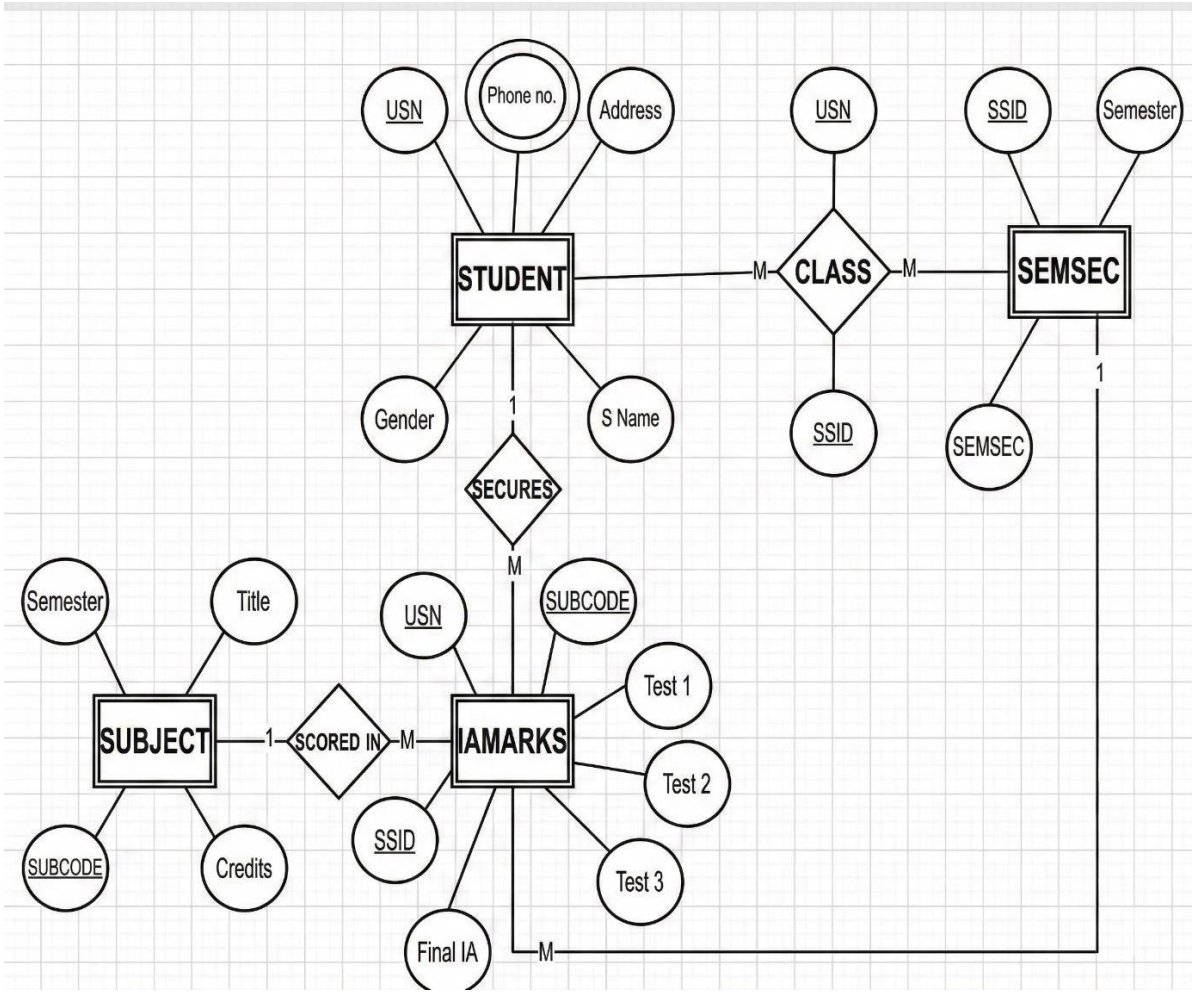
- USN, Subcode, and SSID together form a composite key.
- Test1, Test2, Test3, and FinalIA are normal attributes.

3. Primary keys are represented by underlining the attribute names.
4. **CLASS** is drawn as a relationship (diamond shape) between **STUDENT** and **SEMSEC**.
5. The relationship between **STUDENT and SEMSEC** through CLASS is **M : 1**, meaning many students belong to one semester and section.
6. The **M : N relationship between STUDENT and SUBJECT** is resolved using the associative entity **IAMARKS**.
7. The relationship **STUDENT – IAMARKS** is **1 : M**, indicating one student can have multiple internal assessment records.
8. The relationship **SUBJECT – IAMARKS** is **1 : M**, indicating one subject can have multiple IA records for different students.

9. The relationship **SEMSEC – IAMARKS** is **1 : M**, showing that marks are maintained semester-section wise.

10. All entities, attributes, relationships, primary keys, foreign keys, and cardinalities are clearly represented in the ER diagram.

ER DIAGRAM:



2. Consider following databases and draw ER diagram and convert entities and relationships to relation table for a given scenario:

AIM: To analyze the requirements for a Company Database, design an Entity-Relationship (ER) Diagram representing Employees, Departments, Projects, and their relationships (Works On, Manages, Supervision), and map the ER diagram into a normalized Relational Database Schema.

COMPANY DATABASE:

EMPLOYEE (SSN, Name, Address, Salary, Sex, SuperSSN, DNO)

DEPARTMENT (DNo, DName, MgrSSN, MgrStartDate)

PROJECT (PNo, PName, PLocation, DNO)

DLOCATION (DNo, DLoc)

WORKS_ON (SSN, PNo, Hours)

MANAGES (SSN, DNo)

PROCEDURE:

1. The entities **EMPLOYEE, DEPARTMENT, PROJECT, and DLOCATION** are identified and represented using rectangles.
2. The attributes are represented using ellipses:

EMPLOYEE: SSN (Primary Key), Name, Address, Salary, Sex, SuperSSN, DNO

- SSN is underlined as it is the Primary Key.
- SuperSSN is a foreign key referencing EMPLOYEE (self-relationship).
- DNO is a foreign key referencing DEPARTMENT.

DEPARTMENT: DNo (Primary Key), DName, MgrSSN, MgrStartDate

- DNo is underlined as the Primary Key.
- MgrSSN is a foreign key referencing EMPLOYEE.

PROJECT: PNo (Primary Key), PName, PLocation, DNO

- PNo is underlined as the Primary Key.
- DNO is a foreign key referencing DEPARTMENT.

DLOCATION: DNo, DLoc

- DNo and DLoc together form a composite key.
- DNo is a foreign key referencing DEPARTMENT.

WORKS_ON: SSN, PNo, Hours

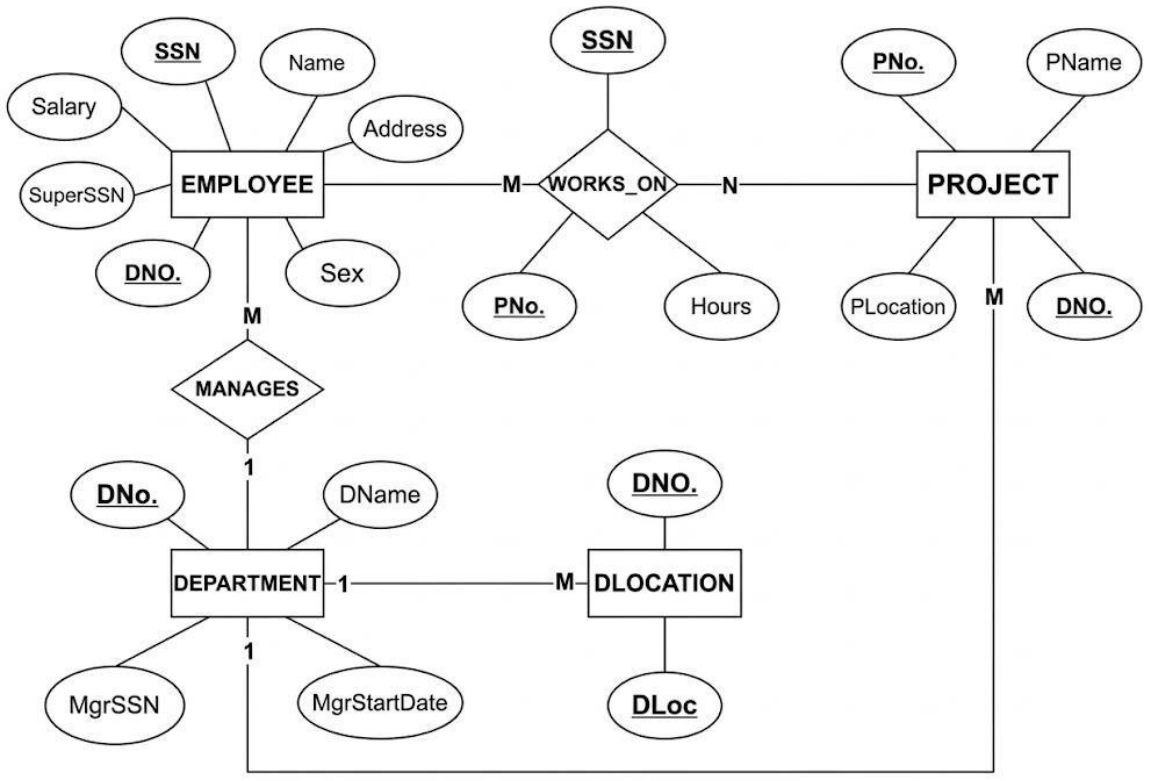
- SSN and PNo together form a composite key.
- Hours is a descriptive attribute of the relationship.

3. Primary keys are represented by underlining the attribute names.
4. The relationship **WORKS_ON** is drawn as a diamond between **EMPLOYEE** and **PROJECT** with **M : N cardinality**, indicating that:
    - One employee can work on many projects.

- One project can have many employees.
- Hours attribute represents the number of hours worked.

5. The relationship **MANAGES** is drawn between **EMPLOYEE** and **DEPARTMENT** with **1 : 1 cardinality**, indicating:
    - One employee manages one department.
    - Each department has one manager.
6. The relationship between **EMPLOYEE and DEPARTMENT** (via DNO) is **M : 1**, meaning:
    - Many employees work in one department.
7. The relationship between **DEPARTMENT and PROJECT** is **1 : M**, meaning:
    - One department controls many projects.
    - Each project belongs to one department.
8. The relationship between **DEPARTMENT and DLOCATION** is **1 : M**, meaning:
    - One department can have multiple locations.
9. A recursive relationship exists in EMPLOYEE through **SuperSSN**, representing that:
    - One employee can supervise many employees.
10. All entities, attributes, relationships, primary keys, foreign keys, and cardinalities are clearly represented in the ER diagram.

ER diagram:



3. Database Schema for a customer-sale scenario

Customer (Cust id: integer, cust_name: string)

Item (item_id: integer, item_name: string, price: integer)

Sale (bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)

For the above schema, perform the following—

1. Create the tables with the appropriate integrity constraints.
2. Insert around 10 records in each of the tables.
3. List all the bills for the current date with the customer names and item numbers.
4. List the total Bill details with the quantity sold, price of the item and the final amount.
5. List the details of the customer who have bought a product which has a price>200.
6. Give a count of how many products have been bought by each customer
7. Give a list of products bought by a customer having cust_id as 5.
8. List the item details which are sold as of today.
9. Create a view which lists out the bill_no, bill_date, cust_id, item_id, price, qty_sold, amount.
10. Create a view which lists the daily sales date wise for the last one week

Sol:

```
DROP DATABASE CUSTOMER_SALE;
CREATE DATABASE CUSTOMER_SALE;
USE CUSTOMER_SALE;
CREATE TABLE CUSTOMER(
CUST_ID INT PRIMARY KEY,
CUST_NAME VARCHAR(100)
);
CREATE TABLE ITEM (
ITEM_ID INT PRIMARY KEY,
ITEM_NAME VARCHAR(100),
PRICE INT);
CREATE TABLE SALE (
BILL_NO INT ,
BILL_DATA DATE,
CUST_ID INT,
ITEM_ID INT,
QTY_SOLD INT ,
FOREIGN KEY (CUST_ID) REFERENCES CUSTOMER(CUST_ID),
FOREIGN KEY (ITEM_ID) REFERENCES ITEM(ITEM_ID)
);
INSERT INTO
CUSTOMER(CUST_ID ,CUST_NAME)
VALUES
(1,'Divesh'),
(2,'Virat'),
(3,'Ankit'),
(4,'Deepak'),
(5,'Hridyanshu'),
(6,'Ashok'),
(7,'Alok'),
(8,'Aman'),
```

```
(9,'Ashish'),
(10,'Mohit');
SELECT * FROM CUSTOMER;

INSERT INTO ITEM VALUES
(101,'Pen',50),
(102,'Notebook',120),
(103,'Bag',800),
(104,'Bottle',250),
(105,'Shoes',1500),
(106,'Watch',2200),
(107,'Mouse',500),
(108,'Keyboard',700),
(109,'Charger',300),
(110,'Headphones',1800);
SELECT * FROM ITEM;

 INSERT INTO SALE VALUES
(1,CURDATE(),1,103,1),
(2,CURDATE(),2,101,3),
(3,CURDATE(),2,105,2),
(4,'2026-02-15',3,104,1),
(5,'2026-02-14',4,106,1),
(6,'2026-02-13',5,102,4),
(7,CURDATE(),6,110,1),
(8,'2026-02-16',7,108,2),
(9,CURDATE(),5,109,5),
(10,'2026-02-12',8,107,1);
SELECT * FROM SALE;
```

```
SELECT S.BILL_NO, C.CUST_NAME, S.ITEM_ID
FROM SALE S
JOIN CUSTOMER C ON S.CUST_ID = C.CUST_ID
WHERE S.BILL_DATA = CURDATE();

SELECT
    S.BILL_NO,
    C.CUST_NAME,
    I.ITEM_NAME,
    S.QTY_SOLD,
    I.PRICE,
    (S.QTY_SOLD * I.PRICE) AS FINAL_AMOUNT
FROM SALE S
JOIN CUSTOMER C ON S.CUST_ID = C.CUST_ID
JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID;

SELECT DISTINCT C.*
FROM CUSTOMER C
JOIN SALE S ON C.CUST_ID = S.CUST_ID
JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID
WHERE I.PRICE > 200;

SELECT C.CUST_NAME, SUM(S.QTY_SOLD) AS TOTAL_PRODUCTS_BOUGHT
FROM CUSTOMER C
JOIN SALE S ON C.CUST_ID = S.CUST_ID
GROUP BY C.CUST_NAME;

SELECT I.ITEM_NAME, S.QTY_SOLD
FROM SALE S
JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID
WHERE S.CUST_ID = 5;
```

```
SELECT I.*
FROM SALE S
JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID
WHERE S.BILL_DATA = CURDATE();


CREATE VIEW Bill_Details AS
SELECT
    S.BILL_NO,
    S.BILL_DATA,
    S.CUST_ID,
    S.ITEM_ID,
    I.PRICE,
    S.QTY_SOLD,
    (S.QTY_SOLD * I.PRICE) AS AMOUNT
FROM SALE S
JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID;

SELECT * FROM BILL_DETAILS;


CREATE VIEW WEEKLY_SALES AS
SELECT
    BILL_DATA,
    SUM(S.QTY_SOLD * I.PRICE) AS TOTAL_DAILY_SALES
FROM SALE S
JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID
WHERE BILL_DATA >= CURDATE() - INTERVAL 7 DAY
GROUP BY BILL_DATA;

SELECT * FROM WEEKLY_SALES;
```

OUTPUT:

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

CUST_ID	CUST_NAME
1	Divesh
2	Virat
3	Ankit
4	Deepak
5	Hridyanshu
6	Ashok
7	Alok
8	Aman
9	Ashish
10	Mohit
NULL	NULL

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

ITEM_ID	ITEM_NAME	PRICE
101	Pen	50
102	Notebook	120
103	Bag	800
104	Bottle	250
105	Shoes	1500
106	Watch	2200
107	Mouse	500
108	Keyboard	700
109	Charger	300
110	Headphones	1800
NULL	NULL	NULL

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

BILL_NO	BILL_DATA	CUST_ID	ITEM_ID	QTY_SOLD
1	2026-03-02	1	103	1
2	2026-03-02	2	101	3
3	2026-03-02	2	105	2
4	2026-02-15	3	104	1
5	2026-02-14	4	106	1
6	2026-02-13	5	102	4
7	2026-03-02	6	110	1
8	2026-02-16	7	108	2
9	2026-03-02	5	109	5
10	2026-02-12	8	107	1

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

BILL_NO	CUST_NAME	ITEM_ID
1	Divesh	103
2	Virat	101
3	Virat	105
7	Ashok	110
9	Hridyanshu	109

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

BILL_NO	CUST_NAME	ITEM_NAME	QTY_SOLD	PRICE	FINAL_AMOUNT
1	Divesh	Bag	1	800	800
2	Virat	Pen	3	50	150
3	Virat	Shoes	2	1500	3000
4	Ankit	Bottle	1	250	250
5	Deepak	Watch	1	2200	2200
6	Hridyanshu	Notebook	4	120	480
7	Ashok	Headphones	1	1800	1800
8	Alok	Keyboard	2	700	1400
9	Hridyanshu	Charger	5	300	1500
10	Aman	Mouse	1	500	500

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CUST_ID	CUST_NAME
1	Divesh
3	Ankit
2	Virat
4	Deepak
8	Aman
7	Alok
5	Hridyanshu
6	Ashok

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CUST_NAME	TOTAL_PRODUCTS_BOUGHT
Divesh	1
Virat	5
Ankit	1
Deepak	1
Hridyanshu	9
Ashok	1
Alok	2
Aman	1

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ITEM_NAME	QTY_SOLD
Notebook	4
Charger	5

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ITEM_ID	ITEM_NAME	PRICE
103	Bag	800
101	Pen	50
105	Shoes	1500
110	Headphones	1800
109	Charger	300

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

BILL_NO	BILL_DATA	CUST_ID	ITEM_ID	PRICE	QTY_SOLD	AMOUNT
1	2026-03-02	1	103	800	1	800
2	2026-03-02	2	101	50	3	150
3	2026-03-02	2	105	1500	2	3000
4	2026-02-15	3	104	250	1	250
5	2026-02-14	4	106	2200	1	2200
6	2026-02-13	5	102	120	4	480
7	2026-03-02	6	110	1800	1	1800
8	2026-02-16	7	108	700	2	1400
9	2026-03-02	5	109	300	5	1500
10	2026-02-12	8	107	500	1	500

Result Grid
Form Editor
Field Types

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

BILL_DATA	TOTAL_DAILY_SALES
2026-03-02	7250

Result Grid
Form Editor
Field Types

Experiment 4.1 :

Question : Consider the following table: Table: CLASS (ID, Name)

```sql
CREATE DATABASE DIVESH;
USE DIVESH;

CREATE TABLE CLASS(
ID INT,
NAME VARCHAR(100)
);
INSERT INTO CLASS(ID ,NAME)
VALUES
(1,'Divesh'),
(2,'Lucky'),
(3,'Ankit'),
(4,'Aryan'),
(5,'Prince'),
(6,'Ashok'),
(7,'Anirudh'),
(8,'Aman'),
(9,'Ashish'),
(10,'Mohit');
SELECT * FROM CLASS;
```

1.SET AUTOCOMMIT = 0; and START TRANSACTION;

```sql
SELECT * FROM CLASS;
SET autocommit = 0;
```

2.INSERT INTO class VALUES (11,'Rahul');

```sql
INSERT INTO CLASS VALUES (11,'RAHUL');
SELECT * FROM CLASS;
```

ID	NAME
1	Divesh
2	Lucky
3	Ankit
4	Aryan
5	Prince
6	Ashok
7	Anirudh
8	Aman
9	Ashish
10	Mohit
11	RAHUL

3.UPDATE class SET name = ‘Abhijeet’ WHERE id= ‘5’;

```
SET SQL_SAFE_UPDATES = 0;
UPDATE CLASS SET NAME = 'ABHIJEET' WHERE ID = 5;
SELECT * FROM CLASS;
```

ID	NAME
1	Divesh
2	Lucky
3	Ankit
4	Aryan
5	ABHIJEET
6	Ashok
7	Anirudh
8	Aman
9	Ashish
10	Mohit
11	RAHUL

4. SAVEPOINT A:

```
SAVEPOINT A;
```

5. INSERT INTO class VALUES (12, ‘Chris’);

6. SAVEPOINT B:

```
SAVEPOINT A;
INSERT INTO CLASS VALUES (12,'CHRIS');
SAVEPOINT B;
```

7. INSERT INTO class VALUES (13, ‘Bravo’);

```
INSERT INTO CLASS VALUES (13,'BRAVO');
SAVEPOINT C;
```

ID	NAME
1	Divesh
2	Lucky
3	Ankit
4	Aryan
5	ABHIJEET
6	Ashok
7	Anirudh
8	Aman
9	Ashish
10	Mohit
11	RAHUL
12	CHRIS
13	BRAVO

8.SAVEPOINT C ;

```
32 •    SAVEPOINT C;
```

9. ROLLBACK TO B;

```
34 •    ROLLBACK TO B;
```

10.ROLLBACK TO A;

```
36 •    ROLLBACK TO A;
```

ID	NAME
1	Divesh
2	Lucky
3	Ankit
4	Aryan
5	ABHIJEET
6	Ashok
7	Anirudh
8	Aman
9	Ashish
10	Mohit
11	RAHUL

11.COMMIT;

```
38 •    COMMIT;
```

Experiment : 4.2

Consider the following table :

Table :

ACCOUNT_TXN Acc_ID, Holder_Name, Balance

101	**Divesh**	**5000**
102	**Lucky**	**7000**
103	**Utsah**	**6000**

ACC_ID	HOLDER_NAME	BALANCE
101	Divesh	5000
102	Lucky	7000
103	Utsah	6000

Perform the following TCL-based SQL script and write the output after each SELECT statement:

1.INSERT INTO ACCOUNT_TXN VALUES (104, 'Prince', 4000);

ACC_ID	HOLDER_NAME	BALANCE
101	Divesh	5000
102	Lucky	7000
103	Utsah	6000
104	PRINCE	4000

2.SAVEPOINT S1 ;

```
SET autocommit = 0;
START TRANSACTION;
SAVEPOINT S1;
```

3. UPDATE ACCOUNT_TXN SET Balance = Balance + 1500 WHERE Acc_ID = 101;

```
UPDATE _ACCOUNT_TXN SET BALANCE = BALANCE+1500 WHERE ACC_ID = 101;
SELECT * FROM ACCOUNT_TXN;
```

ACC_ID	HOLDER_NAME	BALANCE
101	DIVESH	6500
102	LUCKY	7000
103	UTSAH	6000
104	PRINCE	4000

4.UPDATE ACCOUNT_TXN SET Balance = Balance - 1000 WHERE Acc_ID = 102;

```
UPDATE ACCOUNT_TXN SET BALANCE = BALANCE - 1000 WHERE ACC_ID = 102;
SELECT * FROM ACCOUNT_TXN;
```

ACC_ID	HOLDER_NAME	BALANCE
101	DIVESH	6500
102	LUCKY	6000
103	UTSAH	6000
104	PRINCE	4000

5.SAVEPOINT S2;

```
SAVEPOINT S2;
```

6. DELETE FROM ACCOUNT_TXN WHERE Acc_ID = 103;

```
DELETE FROM ACCOUNT_TXN WHERE ACC_ID = 103;
SELECT * FROM ACCOUNT_TXN;
```

ACC_ID	HOLDER_NAME	BALANCE
101	DIVESH	6500
102	LUCKY	6000
104	PRINCE	4000

7. INSERT INTO ACCOUNT_TXN VALUES (105, 'VIRAT', 3000);

```
INSERT INTO ACCOUNT_TXN
VALUES
(105,'VIRAT',3000);

SELECT * FROM ACCOUNT_TXN;
```

ACC_ID	HOLDER_NAME	BALANCE
101	DIVESH	6500
102	LUCKY	6000
104	PRINCE	4000
105	VIRAT	3000

8. SAVEPOINT S3;

```
SAVEPOINT S3;
```

9. UPDATE ACCOUNT_TXN SET Holder_Name = 'LUCKY KUMAR' WHERE Acc_ID = 102;

```
UPDATE ACCOUNT_TXN SET HOLDER_NAME = 'LUCKY KUMAR' WHERE ACC_ID = 102;
SELECT * FROM ACCOUNT_TXN;
```

ACC_ID	HOLDER_NAME	BALANCE
101	DIVESH	6500
102	LUCKY KUMAR	6000
104	PRINCE	4000
105	VIRAT	3000

10. ROLLBACK TO S2;

```
ROLLBACK TO S2;
```

ACC_ID	HOLDER_NAME	BALANCE
101	DIVESH	6500
102	LUCKY	6000
103	UTSAH	6000
104	PRINCE	4000

11. INSERT INTO ACCOUNT_TXN VALUES (106, 'Farah', 4500);

```
INSERT INTO ACCOUNT_TXN
VALUES
(106,'FARAH',4500);

SELECT * FROM ACCOUNT_TXN;
```

ACC_ID	HOLDER_NAME	BALANCE
101	DIVESH	6500
102	LUCKY	6000
103	UTSAH	6000
104	PRINCE	4000
106	FARAH	4500

12. SAVEPOINT S4;

```
SAVEPOINT S4;
```

13. UPDATE ACCOUNT_TXN SET Balance = Balance + 500 WHERE Acc_ID IN (101,104); SELECT * FROM ACCOUNT_TXN;

```
UPDATE ACCOUNT_TXN SET BALANCE = BALANCE + 500 WHERE ACC_ID IN (101,104);
```

ACC_ID	HOLDER_NAME	BALANCE
101	DIVESH	7000
102	LUCKY	6000
103	UTSAH	6000
104	PRINCE	4500
106	FARAH	4500

14. ROLLBACK TO S4;

SELECT * FROM ACCOUNT_TXN;

```
ROLLBACK TO S4;
SELECT * FROM ACCOUNT_TXN;
```

ACC_ID	HOLDER_NAME	BALANCE
101	DIVESH	6500
102	LUCKY	6000
103	UTSAH	6000
104	PRINCE	4000
106	FARAH	4500

15. COMMIT;

```
COMMIT;
```

16. Identify which rows are permanently saved after COMMIT;

ACC_ID	HOLDER_NAME	BALANCE
101	DIVESH	6500
102	LUCKY	6000
103	UTSAH	6000
104	PRINCE	4000
106	FARAH	4500